Parallel Image Stitching

Performance Analysis of Concurrent Architectures in Python

Davide Lombardi

July 20, 2026

UNIVERSITÀ
DEGLI STUDI
FIRENZE

Da un secolo, oltre.

Introduction

o

- **Image Stitching:** combining multiple overlapping images into a single high-resolution panorama
- **Applications:** geospatial mapping, autonomous driving, virtual reality, consumer photography
- **The Problem:** the standard sequential pipeline scales poorly as resolution and dataset size grow
- **Objective:** evaluate Python concurrency paradigms against two language-specific bottlenecks:
  - The **Global Interpreter Lock (GIL)** restricts true multi-threaded execution
  - **Inter-Process Communication (IPC)** introduces serialization overhead

Visual Overview

o

- **Dataset:** OpenDroneMap aerial imagery, downsampled to 50% resolution
- **Sliding window:** panoramas built from non-overlapping windows of 4 images at a time
- **Six architectures** benchmarked on this exact pipeline against a sequential baseline

The Stitching Pipeline: Feature Processing

o

1. **Feature Extraction (SIFT):**
   - Detects scale-space extrema via the Difference of Gaussians
   - Runs independently per image, so it is embarrassingly parallel
2. **Feature Matching (FLANN):**
   - Compares 128-dimensional descriptors between adjacent frames
   - Lowe's ratio test with $\tau < 0.7$ filters out spurious matches
3. **Homography Estimation (RANSAC):**
   - Iteratively fits a 3×3 matrix H that separates inliers from outliers
   - Runtime is non-deterministic and depends on match quality

The Stitching Pipeline: Merging & the Structural Bottleneck

o

Warping & Blending

- Linear alpha blending removes visible seams between frames
- Heavy, vectorizable array manipulation

Feature Re-extraction

- Must run again on the updated canvas after every merge
- Canvas area grows monotonically: $A_n \approx A_0 + \sum_{i=1}^{n} \Delta A_i$
- **Strict dependency:** stitch i cannot start until stitch $i-1$ is committed

This rigid, monotonically-growing sequential chain is the central obstacle every parallel architecture in this work has to work around

Targeted Parallel Pipeline

o

Addresses each bottleneck according to its computational nature.

- **Threaded I/O:** image loading uses a `ThreadPoolExecutor`, since file reads release the GIL
- **Process-pool SIFT:** CPU-bound extraction uses independent processes to fully bypass the GIL
  - Requires a pickling workaround. C++ `cv2.KeyPoint` objects are decomposed into plain tuples before crossing the process boundary
- **Tiled Blending:** the output canvas is split into horizontal tiles blended concurrently
  - NumPy array arithmetic releases the GIL, so threads are highly efficient here too

Producer-Consumer Paradigm

o

Decouples the independent extraction work from the sequential fold:

- **Temporal overlap:** SIFT extraction for image $i + 1$ runs concurrently with the fold of image i
- **Architecture:** a producer thread dispatches every image to a warm process pool and pushes results onto a bounded `queue.Queue`, strictly in order
- **Zero-copy handoff:** producer and consumer share the same process address space, so passing an item costs only an object reference
- **Backpressure:** a fixed queue depth blocks the producer if it runs ahead of the consumer. This caps memory usage

MapReduce Paradigm

o

Restructures the fold into a binary merge tree:

- **Map phase:** SIFT extraction for all n images runs concurrently
- **Reduce phase:** adjacent pairs such as $(0, 1)$ and $(2, 3)$ are stitched concurrently. The process repeats over $\lceil \log_2 n \rceil$ levels instead of $n - 1$ sequential steps
- **The trade-off:** pairs are fixed by index rather than verified spatial overlap. This yields the highest peak speedup, but the result becomes geometrically unstable when a fixed pairing does not actually overlap

IPC Optimization Strategies

o

Pickling roughly 13.5 MB image arrays across process boundaries is expensive

- **Joblib:** the `loky` backend keeps worker processes alive across calls and automatically memory-maps arrays larger than 1 MB to disk instead of pickling them
- **Shared Memory:** images are copied once into a `multiprocessing.shared_memory` block. Workers then reconstruct a NumPy view instead of receiving a pickled copy
  - Only the outbound transfer is optimized. Results still return via the normal pickled channel

Performance Results

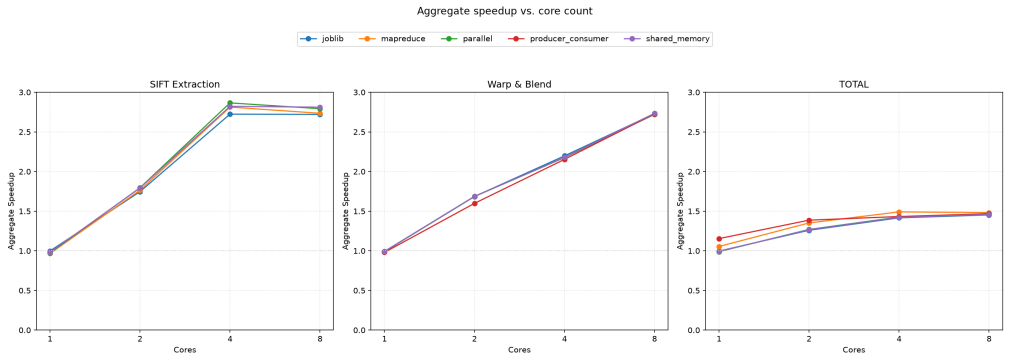
o

UNIVERSITÀ DEGLI STUDI FIRENZE
Da un secolo, oltre.

UNIVERSITÀ DEGLI STUDI FIRENZE
Da un secolo, oltre.

Results: The Amdahl Ceiling

o

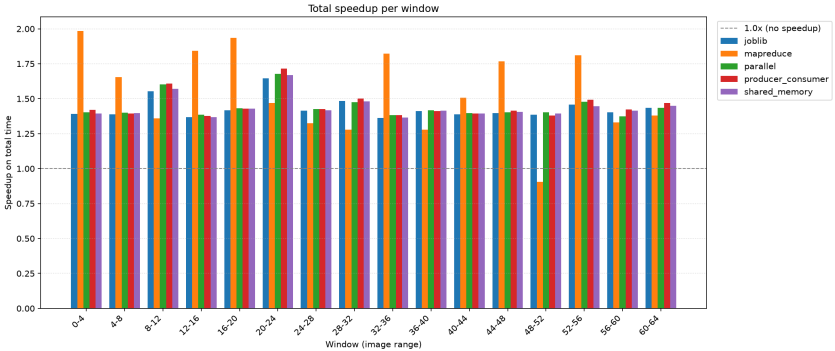


- **SIFT Extraction** plateaus at roughly 2.7–2.9× past 4 cores, due to task starvation with a small window
- **Warp & Blend** scales almost linearly to 8 cores. This memory-bound work benefits from hyper-threading
- **TOTAL never exceeds roughly 1.5×.** Feature re-extraction takes about 65% of total time and stays fixed at approximately 14.5 s: it is never parallelized by any architecture

UNIVERSITÀ DEGLI STUDI FIRENZE
Da un secolo, oltre.

Results: Stability & the MapReduce Trade-off

o



- **Linear-fold pipelines** are highly stable at 1.3× to 1.7× on every window
- **MapReduce** peaks near 2.0× but drops to 0.91×
- **Why:** tree pairing is fixed by index rather than verified spatial overlap, so a geometrically unlucky pair has no fallback

Key Numbers

o

Isolated Phases

- SIFT Extraction peaks at **2.9**× at 4 cores, then plateaus
- Warp & Blend is still rising at 8 cores, reaching **2.7**×

End-to-End

- TOTAL speedup is capped at **roughly 1.5**×
- MapReduce has the lowest average time, 21.1 s, and the highest peak, 2.0×, but is the least predictable
- The four IPC-optimized linear-fold variants differ by **under 1%** from each other

Conclusions

o

- Isolated CPU-bound and I/O-bound tasks overcome the GIL cleanly with targeted `multiprocessing` and `threading`
- **IPC optimization gives diminishing returns** at this scale. Engineering effort on serialization, via Joblib and shared memory, yielded under 1% gain over the simplest process-pool baseline
- **Main Lesson:** true end-to-end scalability needs functional decomposition, as in MapReduce's merge tree, rather than retrofitting concurrency onto a rigid sequence. This comes at the cost of predictability
- **Future Work:** algorithmic restructuring of the re-extraction phase or offloading the sequential chain to GPUs

Parallel Image Stitching

Thank you for listening!
Any questions?

Per-Phase Speedup as Cores Increase

o

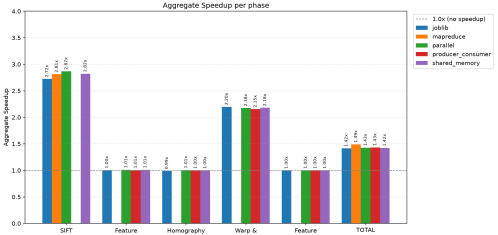


Figure: 4 cores

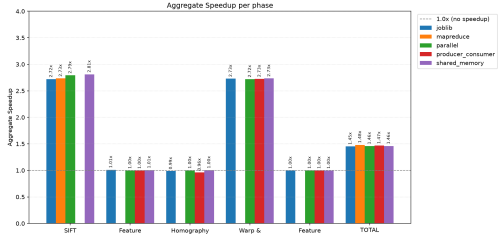


Figure: 8 cores

- Match, Homography and Re-extraction stay flat at about 1.0× at every core count. None of the six pipelines ever parallelizes them
- At 8 cores with window size 4, only 4 tasks feed an 8-worker pool. This looks like a hardware plateau, but is it?

UNIVERSITÀ DEGLI STUDI FIRENZE
Da un secolo, oltre.

Task Starvation, Not a Hardware Limit

o

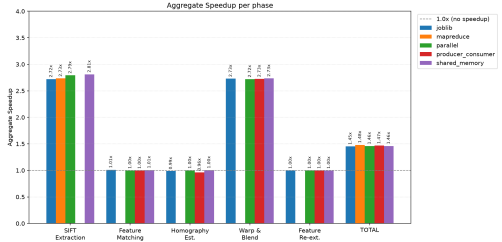


Figure: 8 cores, window size 4

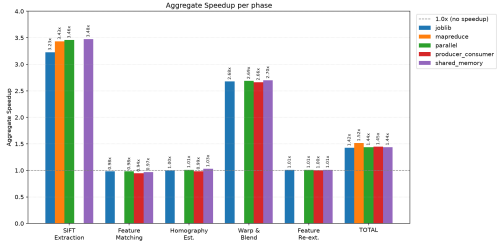


Figure: 8 cores, window size 8

- With window size 4, only 4 tasks feed an 8-worker pool. Half the pool sits idle, and dispatch overhead is disproportionate to the actual computation
- Doubling the window to 8 images fully populates the pool. SIFT Extraction breaks past the 2.7–2.8× plateau and reaches roughly 3.2–3.5×
- **Confirms** that the earlier plateau was task starvation, not a hardware ceiling. Hyper-threading limits still apply, but only once the pool is actually full